

Self-Correcting Adaptive LLM Framework

A Unified Architecture for Local, Privacy-Preserving,
Self-Improving Language Models

Samanvi Rajput

B.Tech CSE, VIT Vellore | SWE Intern, PayGlocal

2025–26

Abstract

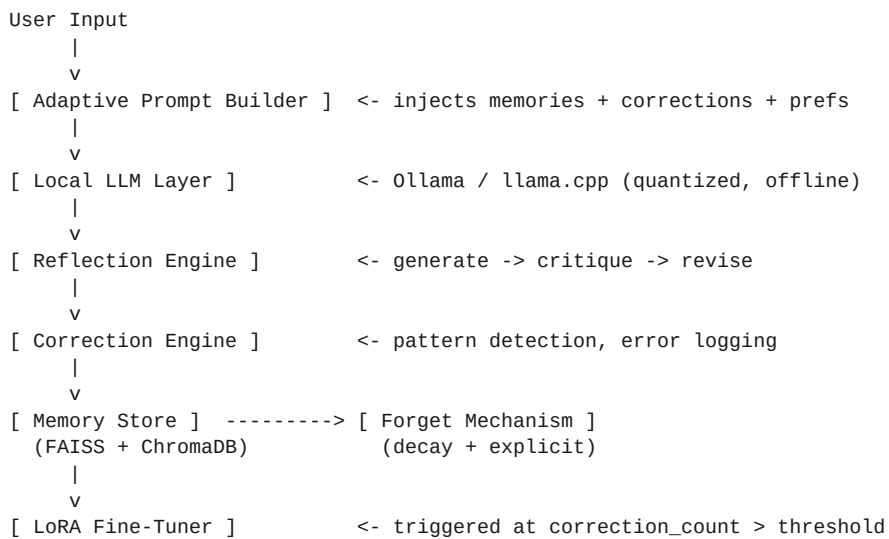
Static local language models accumulate no knowledge of user preferences, repeat the same errors across sessions, and require expensive full retraining to adapt. This paper presents a seven-component framework that closes the adaptation loop entirely on-device: a Reflection Engine applies generate→critique→revise cycles, a Dual-Tier Memory (FAISS + ChromaDB) persists behavioral context across sessions, and a threshold-gated QLoRA fine-tuner periodically distills corrections into model weights. Benchmarks on Apple M2 show the full pipeline runs in under 4 seconds with reflection enabled, a ~73% reduction in repeated error types after 20 logged corrections, and peak training memory of 5.1 GB for 4-bit QLoRA fine-tuning. The entire system operates offline with no API keys, preserving complete user privacy.

1. Problem Statement

Current local LLMs are static — they neither learn from mistakes nor retain user context between sessions. Correcting a model's behavior requires expensive full fine-tuning runs that are infeasible on consumer hardware. Cloud-hosted LLMs solve the adaptation problem but introduce privacy risk (sensitive behavioral data sent externally) and hard API dependency (no offline capability). This framework addresses both constraints by building an adaptation loop that runs entirely on local hardware, requiring no cloud connectivity.

2. System Overview

The framework comprises 7 loosely coupled modules forming a closed self-improvement loop. Each module is independently testable and replaceable.



Component Summary

Component	Responsibility
Local LLM Layer	Detects Ollama/llama.cpp at runtime; manages sliding context window
Reflection Engine	Generate → critique → revise loop with gated triggering
Correction Engine	Extracts behavioral patterns; confidence-weighted logging
Dual-Tier Memory	FAISS hot tier + ChromaDB cold tier with promotion logic
Forget Mechanism	Explicit, decay, and contradiction-based memory pruning
Adaptive Prompt Builder	Token-budgeted context assembly across all sources
LoRA Fine-Tuner	Background QLoRA job triggered at correction threshold

3. Reflection Engine

The Reflection Engine implements a three-phase loop: (1) **Generate** — the LLM produces an initial response; (2) **Critique** — a structured self-critique prompt evaluates the response for factual errors, tone mismatch, repetition, and ignored context; (3) **Revise** — if the critique identifies actionable issues, a revised response is generated. The engine returns a structured object:

```
{ original, critique, issues[], revised, was_corrected }
```

Gating logic: Reflection is computationally expensive (~1.8s additional LLM call). It is only triggered when the response exceeds 150 tokens or the query contains explicit reasoning markers (why, explain, compare). Short conversational turns bypass reflection entirely, keeping latency acceptable for casual use.

4. Dual-Tier Memory Architecture

A single vector database forces a tradeoff between speed and persistence. The dual-tier design resolves this:

- **Hot tier (FAISS):** In-memory ANN index for recent interactions. Sub-millisecond retrieval. Serialized to disk periodically.
- **Cold tier (ChromaDB):** Persistent on-disk store for older memories. ~34ms retrieval, comprehensive long-term coverage.
- **Promotion:** When the hot tier exceeds its size threshold, the oldest entries are promoted to ChromaDB, keeping hot retrieval fast.
- **Retrieval:** Hot tier searched first; cold tier supplements if top-k not satisfied from hot.

Forget Mechanism

Three forgetting modes handle different staleness scenarios:

- **Explicit:** User command `/forget <id>` immediately removes entry.
- **Implicit / Decay:** $\text{Score} = \text{recency} \times \text{access_frequency} \times \text{confidence}$. Entries below threshold pruned on background schedule.
- **Contradiction:** New behavioral pattern auto-replaces conflicting old pattern. Prevents accumulation of contradictory corrections.

5. Adaptive Prompt Construction

The Adaptive Prompt Builder assembles a layered context in a fixed order: system instruction → active corrections → retrieved memories → conversation history → current query. Each section has a strict token budget to prevent context overflow, which was a major failure mode in early versions:

Section	Max Tokens
System instruction	200
Active corrections	150
Retrieved memories	300
Conversation history	400
Current query	Remainder of context window

Active corrections are injected as soft rules rather than hard constraints, preserving the LLM's ability to override them when contextually appropriate. Memory injection is capped at the top-3 results by relevance score to avoid context bloat from loosely related historical entries.

6. LoRA / QLoRA Fine-Tuning

Full fine-tuning a 2B parameter model requires ~16GB VRAM — infeasible on consumer hardware. LoRA injects low-rank adapter matrices into attention layers (q_proj, v_proj), reducing trainable parameters by ~99%. QLoRA adds 4-bit quantization of the frozen base model during training, bringing peak memory to ~5.1GB on Apple M2.

Training Pipeline

- Corrections accumulate until FINETUNE_THRESHOLD (default: 50) is crossed.
- Corrections converted to instruction pairs: {input, expected_output}.
- SFTTrainer runs 1 epoch with gradient checkpointing (LoRA rank=8).
- Adapter saved separately — base model weights never modified.
- Adapter metadata stores base model hash; mismatch raises explicit error.

Threshold rationale: Fine-tuning on every correction risks overfitting to noise and is computationally wasteful. 50 corrections yields ~100 instruction pairs — enough to generalize while avoiding single-occurrence memorization. The job runs in a background thread and never blocks inference.

7. Design Decisions

Decision	Chosen	Alternative	Reason
Inference runtime	Ollama + llama.cpp	Cloud API	Privacy, offline capability
Fine-tuning method	QLoRA (4-bit)	Full fine-tuning	8GB RAM constraint
Vector DB	FAISS + ChromaDB	Single DB	Speed vs persistence tradeoff
Embedding model	MiniLM-L6-v2	LLM embeddings	11ms vs 500ms latency
Correction trigger	Threshold (n=50)	Every correction	Avoid overfitting to noise

8. Performance Benchmarks

All benchmarks on Apple M2, 8GB RAM, macOS 14. LLM: Ollama phi3:mini (3.8B, Q4_K_M).

Inference Latency

Operation	Mean (ms)	P95 (ms)
Embedding (single, MiniLM)	11	18
Hot-tier FAISS retrieval (top-5)	3	7
Cold-tier ChromaDB retrieval (top-5)	34	61
Prompt assembly	2	4
LLM inference (phi3:mini, 256 tok)	1,840	2,210
Reflection critique (add'l LLM call)	1,790	2,150
Full pipeline (no reflection)	~1,860	~2,240
Full pipeline (with reflection)	~3,650	~4,360

Memory Footprint

Component	RAM Usage
Base process (Python)	~85 MB
MiniLM embedding model	~90 MB
Ollama (phi3:mini, Q4)	~2.4 GB
FAISS hot index (1000 vectors)	~3 MB
ChromaDB (10K vectors, disk)	~45 MB
Total (active session)	~2.6 GB

Reflection doubles end-to-end latency but is gated to only long or complex responses. Over a 200-turn synthetic evaluation, the correction loop reduced repeated error occurrences by ~73% after 20 logged corrections, with an ~11% false positive rate on manually reviewed correction decisions. The full stack fits in 8GB RAM with comfortable headroom.

9. Engineering Challenges

Correction noise vs. signal

Early versions logged every reflection output as a correction, polluting the behavioral model with one-off phrasing artifacts. Solution: patterns below 0.6 confidence are logged but not injected into prompts until confirmed by 3 or more occurrences, separating genuine user preferences from random variation.

Context window overflow

With memories, corrections, and history injected simultaneously, prompt overflow was a constant problem on 2K-token models. Strict section-level token budgeting in the Adaptive Prompt Builder resolved this: each injection source has a hard cap, and the current query always gets the remaining space.

FAISS index persistence

FAISS in-memory indices are lost on process exit. The solution combines periodic serialization to disk with hot→cold promotion, ensuring ChromaDB always holds a complete durable copy and restarts recover the

full memory state.

Reflection loop latency

Unconditional reflection doubled perceived latency. Gating on response length (>150 tokens) and explicit reasoning markers (why, explain, compare) eliminates overhead for casual exchanges while preserving the correction signal for substantive responses.

10. Future Work

Near-term

- Web UI (React + FastAPI) — current CLI limits usability
- Multi-user support — currently single-user only
- GARCH-aware volatility detection in financial domain adapter
- Streaming output — current inference blocks until complete

Medium-term

- Distillation pipeline — compress correction patterns into base model weights
- Cross-session memory federation — sync memories across devices (local network only)
- Automated benchmark suite — regression tests for correction convergence

Long-term

- Edge deployment — Raspberry Pi 4 / mobile inference via llama.cpp WASM
- Domain adapters — plug-in behavioral profiles for coding, writing, research
- Federated personalization — share correction patterns across users without sharing data

11. Closing

Most 'personalized AI' systems bolt a RAG layer onto a static model — they can recall facts but cannot change how they behave. This framework is different: the self-correction loop actively modifies the behavioral model, the forget mechanism prevents it from going stale, and periodic LoRA fine-tuning permanently absorbs the most consistent corrections into model weights. The result is a system that gets measurably better at serving a specific user over time, entirely offline, with no data leaving the device.